POOLSTREAM

LAPORAN TUGAS · PROJECT SETUP

Repository, Struktur Folder, Git Workflow & Setup

Studi kasus **PoolStream** — aplikasi manajemen billiard dan lounge berbasis Laravel 13, Inertia.js, Vue 3, dan Bootstrap 5.

41	231	19	18
COMMIT	BERKAS TRACKED	TEST LULUS	MIGRASI

DISUSUN OLEH	Muhammad Anurohim
REPOSITORY	github.com/anurr23/poolstream
BRANCH UTAMA	main · 41 commit · 1 contributor
TANGGAL	28 Oktober 2026

Laravel 13 · Inertia.js · Vue 3 · Bootstrap 5 · Vite · PHPUnit

NAVIGASI DOKUMEN

Daftar Isi

Laporan ini disusun sebagai bukti pemenuhan empat kriteria tugas: repository proyek, struktur folder aplikasi, workflow Git, dan dokumentasi setup.

00 **Ringkasan Pemenuhan Kriteria** 3

01 **Pendahuluan** 4

02 **Repository Proyek** 6

03 **Struktur Folder Aplikasi** 9

04 **Workflow Git** 14

05 **Dokumentasi Setup** 18

06 **Verifikasi dan Kesimpulan** 23

A **Lampiran: Perintah Penting** 25

Peta cepat

Kebutuhan	Lihat
Identitas dan riwayat repository	Bab 2 — Repository Proyek
Peta folder dan alur request	Bab 3 — Struktur Folder Aplikasi
Aturan branch, commit, dan review	Bab 4 — Workflow Git
Langkah instalasi dan troubleshooting	Bab 5 — Dokumentasi Setup
Bukti test, build, dan checklist	Bab 6 — Verifikasi dan Kesimpulan

> **Cara membaca laporan.** Setiap bab memetakan satu kriteria tugas ke bukti yang dapat diperiksa langsung di repository. Angka pada laporan diambil dari kondisi repository per 28 Oktober 2026 (commit `8ae9aa7`).

RINGKASAN EKSEKUTIF

00

Empat Kriteria, Satu Repository yang Siap Dijalankan

PoolStream adalah aplikasi operasional pool hall yang menangani sesi meja billiard, pesanan makanan dan minuman, checkout, cetak struk, serta laporan. Repository-nya publik, riwayat commit-nya dapat ditelusuri, dan seluruh langkah setup didokumentasikan dalam README serta dokumen pendukung.

✓	**Repository proyek** Repository publik `github.com/anurr23/poolstream`, branch `main`, 41 commit, 1 contributor, dan riwayat dari 13 Juli hingga 28 Oktober 2026.	TERPENUHI
✓	**Struktur folder aplikasi** Pemisahan jelas antara backend Laravel (`app/`), frontend Inertia (`resources/js/`), skema database (`database/`), routing (`routes/`), dan pengujian (`tests/`).	TERPENUHI
✓	**Workflow Git** Model branch, Conventional Commits, checklist pull request, aturan line ending, dan kebijakan rilis terdokumentasi di `CONTRIBUTING.md`; 75,6% commit mengikuti convention.	TERPENUHI
✓	**Dokumentasi setup** README memuat prasyarat, quick start, setup manual, konfigurasi environment, database, test, scheduler, relay, dan troubleshooting. Setup dapat diulang dari repository bersih.	TERPENUHI

41	231	12,6k	19	18
COMMIT	BERKAS TRACKED	BARIS KODE	TEST LULUS	MIGRASI

Hasil. Keempat kriteria tugas terpenuhi dan dapat diverifikasi ulang dengan perintah di atas. Repository juga sudah dibersihkan dari dependensi `vendor/` dan file environment lokal yang tidak seharusnya masuk ke Git.

```
# verifikasi cepat dari repository bersih
git clone https://github.com/anurr23/poolstream.git
cd poolstream && composer setup
composer test        # 19 passed, 48 assertions
npm run build        # build produksi berhasil
```

BAB 01 · PENDAHULUAN

01 Konteks, Tujuan, dan Ruang Lingkup

Aplikasi operasional pool hall tidak cukup hanya berjalan di satu komputer. Ia perlu dapat di-clone, dipasang ulang, diuji, dan dikembangkan tanpa menebak-nebak konfigurasi. Karena itu, repository, struktur folder, workflow Git, dan dokumentasi setup diperlakukan sebagai bagian dari produk, bukan pelengkap.

1.1 Latar belakang

PoolStream menangani alur kerja harian kasir dan admin: memulai sesi meja, menambah pesanan F&B, menghitung biaya berjalan, melakukan checkout, mencetak struk, dan membaca laporan pendapatan. Alur tersebut melibatkan banyak entitas—meja, paket tarif, menu, transaksi, item transaksi, dan pengguna—sehingga pemisahan tanggung jawab di dalam kode menjadi penting.

Repository ini dibangun sebagai monolith Laravel dengan Inertia.js dan Vue 3. Pendekatan tersebut menjaga satu basis kode, satu proses deploy, dan satu sumber kebenaran untuk aturan bisnis, tanpa mengorbankan pengalaman antarmuka yang reaktif. Laporan ini mendokumentasikan bagaimana struktur tersebut disusun dan bagaimana proyek dijalankan dari nol.

1.2 Tujuan

1. **Repository proyek.** Menyediakan repository publik dengan riwayat commit yang dapat ditelusuri, branch utama yang jelas, dan konfigurasi repository yang sehat.
2. **Struktur folder aplikasi.** Menyusun direktori berdasarkan tanggung jawab sehingga backend, frontend, database, route, dan test mudah ditemukan.
3. **Workflow Git.** Menetapkan model branch, format commit, checklist review, serta aturan hygiene yang konsisten untuk perubahan berikutnya.
4. **Dokumentasi setup.** Menulis langkah instalasi yang dapat diikuti dari repository bersih, termasuk prasyarat, environment, database, test, dan troubleshooting.

1.3 Ruang lingkup

- Audit repository, riwayat commit, branch, dan berkas yang dilacak Git.
- Pemetaan struktur folder serta alur request dari route hingga halaman Vue.
- Dokumentasi workflow Git dan quality gate yang tersedia di repository.
- Verifikasi setup melalui test suite, build frontend, dan pemeriksaan konfigurasi.
- **Di luar ruang lingkup:** deployment produksi, integrasi relay USB nyata, audit keamanan menyeluruh, dan pengujian beban.

1.4 Metode pengerjaan

Tahap	Aktivitas	Keluaran
Audit repository	`git log`, `git ls-files`, `git fsck`, pemeriksaan remote dan branch.	Statistik commit, daftar berkas, temuan hygiene.
Audit struktur	Penelusuran direktori `app/`, `resources/js/`, `database/`, `routes/`, dan `tests/`.	Peta folder dan tabel tanggung jawab.
Verifikasi	`php artisan test` dan `npm run build` pada environment terisolasi.	19 test lulus, build frontend berhasil.
Dokumentasi	Penyusunan README, `CONTRIBUTING.md`, dan laporan PDF ini.	Dokumen setup dan workflow yang dapat diikuti.

1.5 Profil aplikasi

Nama aplikasi	PoolStream — Billiard & Lounge Management System
Domain	Operasional pool hall: sesi meja, F&B, checkout, struk, laporan.
Stack	Laravel 13, PHP 8.3+, Inertia.js, Vue 3, Bootstrap 5, Vite 8, PHPUnit 12.
Database	SQLite (default), MySQL, atau PostgreSQL.
Repository	`https://github.com/anurr23/poolstream` (publik).
Branch utama	`main` — 41 commit, 231 berkas tracked, 1 contributor.
Status	Aplikasi berjalan, test suite lulus, build frontend berhasil.

1.6 Istilah teknis

Istilah	Makna dalam laporan ini
Repository	Tempat penyimpanan kode beserta riwayat perubahannya, di-host di GitHub.
Branch	Garis pengembangan terpisah; `main` adalah branch utama.
Commit	Satu unit perubahan yang disimpan bersama pesan dan metadata penulis.
Inertia	Jembatan antara controller Laravel dan halaman Vue tanpa API terpisah.
Migrasi	Berkas PHP yang mendefinisikan perubahan skema database secara versioned.

> **Cara membaca angka.** Statistik pada laporan ini dihitung dari commit `8ae9aa7` per 28 Oktober 2026. Perubahan setelah tanggal tersebut tidak tercermin dalam laporan.

BAB 02 · REPOSITORY PROYEK

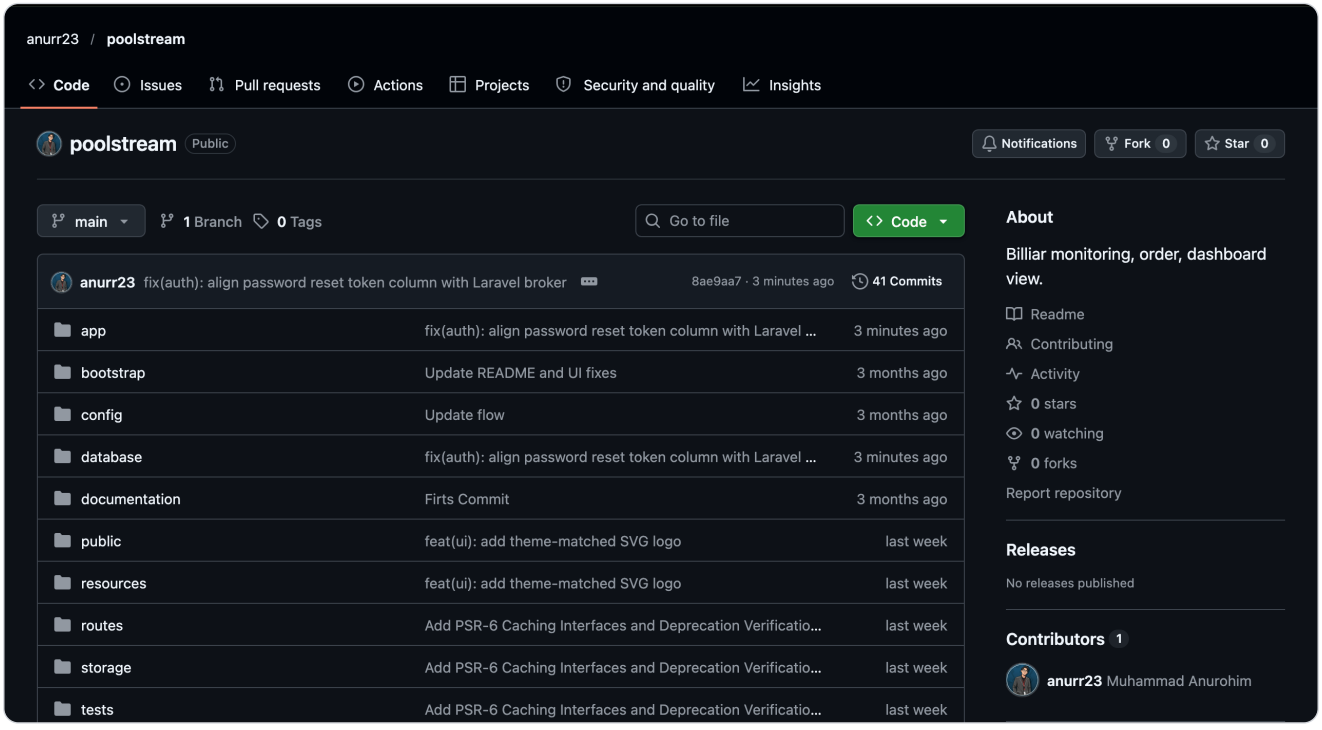
02

Identitas, Riwayat, dan Kebersihan Repository

Repository adalah titik masuk pertama bagi dosen, reviewer, atau developer lain. Bab ini merangkum identitas repository, aktivitas commit, komposisi berkas, dan perbaikan hygiene yang dilakukan agar repository tidak membawa dependensi atau rahasia lokal.

2.1 Identitas repository

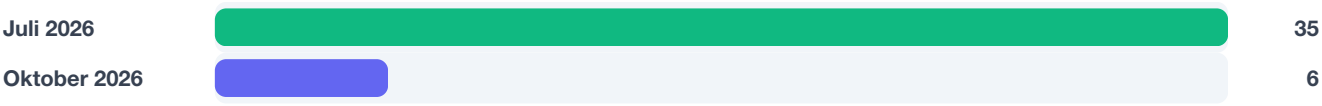
URL	`https://github.com/anurr23/poolstream`
Visibilitas	Publik — dapat diperiksa tanpa akses khusus.
Pemilik	`anurr23` (Muhammad Anurohim).
Branch default	`main` ; jumlah branch saat ini 1.
Tag rilis	Belum ada; kebijakan tag annotated didokumentasikan di `CONTRIBUTING.md` .
Commit	41 commit, 1 contributor, 0 pull request terbuka, 0 issue terbuka.
Rentang riwayat	13 Juli 2026 — 28 Oktober 2026.
Lisensi	MIT (dicantumkan pada README).



Gambar 2.1. Halaman repository publik PoolStream. Terlihat branch `main` , 41 commit, 1 contributor, dan struktur tingkat atas tanpa folder `vendor/` .

2.2 Aktivitas commit

Riwayat commit menunjukkan dua periode kerja: pengembangan awal yang intensif pada Juli 2026 dan penyempurnaan UI serta perbaikan pada Oktober 2026. Distribusi ini wajar untuk proyek akademik: fondasi domain dan tampilan dibangun lebih dulu, lalu diikuti iterasi desain dan perbaikan bug.



Commit terbaru juga menunjukkan perawatan repository, bukan hanya penambahan fitur: pembersihan dependensi yang tidak seharusnya dilacak, perbaikan alur reset password, dan pembaruan dokumentasi. Detail commit dibahas pada Bab 4.

2.3 Statistik kontribusi

Metrik	Nilai	Keterangan
Total commit	41	Semua oleh satu contributor.
Berkas tracked	231	Tidak termasuk `vendor/` dan `node_modules/`.
Baris kode aplikasi	12.670	3.182 PHP, 6.950 Vue/JS, 2.538 SCSS.
Perubahan baris	+35.194 / −6.253	Akumulasi commit, tanpa `vendor/` dan `.env`.
Controller / model	14 / 6	Termasuk controller autentikasi dan controller dasar.
Halaman Vue / komponen	18 / 18	Halaman Inertia dan komponen bersama.
Migrasi / test class	18 / 8	19 method test, 48 assertion.

2.4 Struktur tingkat atas

Direktori	Isi	Status Git
`app/`	Controller, middleware, request validation, model, command, dan service domain.	Tracked
`bootstrap/`	Bootstrap aplikasi, registrasi middleware, dan jadwal scheduler.	Tracked
`config/`	Konfigurasi Laravel (auth, database, cache, queue, session, filesystem).	Tracked
`database/`	Migrasi, factory, dan seeder data demo.	Tracked
`documentation/`	Diagram instalasi relay dan gambar referensi perangkat.	Tracked
`public/`	Front controller, aset publik, logo, dan hasil build Vite.	Tracked
`resources/`	Halaman Vue, komponen, layout, SCSS, dan Blade root.	Tracked
`routes/`	`web.php`, `auth.php`, dan `console.php`.	Tracked
`storage/`	Log, upload foto, gambar F&B, dan cache framework.	Tracked (struktur)

Direktori	Isi	Status Git
`tests/`	Test fitur dan unit dengan PHPUnit.	Tracked
`vendor/`	Dependensi Composer; dapat dipasang ulang dari `composer.lock`.	Ignored
`node_modules/`	Dependensi npm; dapat dipasang ulang dari `package-lock.json`.	Ignored

2.5 Kebersihan repository

Audit menemukan dua artefak yang tidak seharusnya dilacak: folder `vendor/` (9.595 berkas) dan file `.env` yang berisi konfigurasi lokal. Keduanya dihapus dari indeks Git pada commit `25c40a8` tanpa menghapus salinan di disk, lalu `/vendor` ditambahkan ke `.gitignore`.

```
# commit pembersihan repository
git show --stat 25c40a8
# 9.600 berkas berubah: .env dan vendor/ keluar dari indeks

# verifikasi bahwa tidak ada artefak lokal yang masih dilacak
git ls-files | grep -E '^(vendor/|\.env$)'   # tidak ada output
```

Catatan keamanan. File `.env` tetap ada di riwayat Git lama karena penghapusan dari indeks tidak menghapus blob historis. Untuk penggunaan produksi, rotasi `APP_KEY` dan kredensial database, lalu bersihkan riwayat dengan alat seperti `git filter-repo` bila repository pernah memuat rahasia nyata.

2.6 Dampak pembersihan

Pembersihan indeks Git tidak mengubah kode aplikasi, tetapi mengubah cara repository di-clone dan dipasang. Dependensi kini dipasang dari lock file, bukan disalin dari mesin pengembang.

Metrik	Sebelum	Sesudah
Berkas tracked	9.826	231
Folder `vendor/`	Ikut di Git	Ignored; dipasang Composer
File `.env`	Ikut di Git	Ignored; dibuat dari `.env.example`
Ukuran clone	Besar	Ringan dan reproducible

> **Hasil.** Repository kini hanya memuat sumber aplikasi, konfigurasi contoh, migration, dan test. Artefak yang dapat dihasilkan ulang tidak lagi masuk ke riwayat commit baru.

BAB 03 · STRUKTUR FOLDER

03

Pemisahan Tanggung Jawab dari Route hingga Vue

Struktur folder mengikuti pola MVC Laravel, lalu diperluas dengan lapisan service untuk aturan bisnis dan direktori Inertia untuk antarmuka. Tujuannya sederhana: setiap perubahan harus punya tempat yang jelas, sehingga reviewer tidak perlu menebak di mana logika berada.

3.1 Prinsip penyusunan

- **Satu tanggung jawab per direktori.** Controller mengatur request/response; service menghitung aturan bisnis; model mengelola data; Vue menangani tampilan.
- **Monolith Inertia.** Tidak ada API terpisah untuk halaman internal; controller mengembalikan komponen Inertia dan props yang dibutuhkan.
- **Domain dekat dengan datanya.** Perhitungan biaya sesi dan F&B dipusatkan di `TransactionService`, bukan tersebar di banyak controller.
- **Test di samping kode.** Factory dan test disimpan di `database/factories` dan `tests/` agar perilaku mudah diverifikasi.
- **Aset melalui Vite.** SCSS dan JavaScript masuk lewat `resources/`, lalu dikompilasi ke `public/build/`.

3.2 Peta folder

```
poolstream/
├── app/
│   ├── Http/              # controller, middleware, request
│   ├── Models/            # 6 model Eloquent
│   ├── Services/          # TransactionService, relay bridge
│   └── Console/Commands/  # auto-checkout
├── bootstrap/             # bootstrap + scheduler
├── config/                # konfigurasi Laravel
├── database/
│   ├── factories/         # factory test
│   ├── migrations/        # 18 migrasi
│   └── seeders/           # data demo
├── documentation/         # diagram relay
├── public/
│   ├── build/             # hasil Vite
│   └── images/            # logo
├── resources/
│   ├── js/
│   │   ├── Components/    # komponen bersama
│   │   ├── Layouts/       # admin, kasir, guest
│   │   ├── Pages/         # 18 halaman
│   │   └── utils/         # receipt.js
│   ├── scss/              # tema bb-
│   └── views/             # app.blade.php
├── routes/                # web, auth, console
├── storage/               # log, upload, cache
├── tests/                 # Feature + Unit
├── composer.json          # dependensi PHP
├── package.json           # dependensi frontend
├── vite.config.js         # Vite + Vue + SCSS
└── README.md              # setup + workflow
```

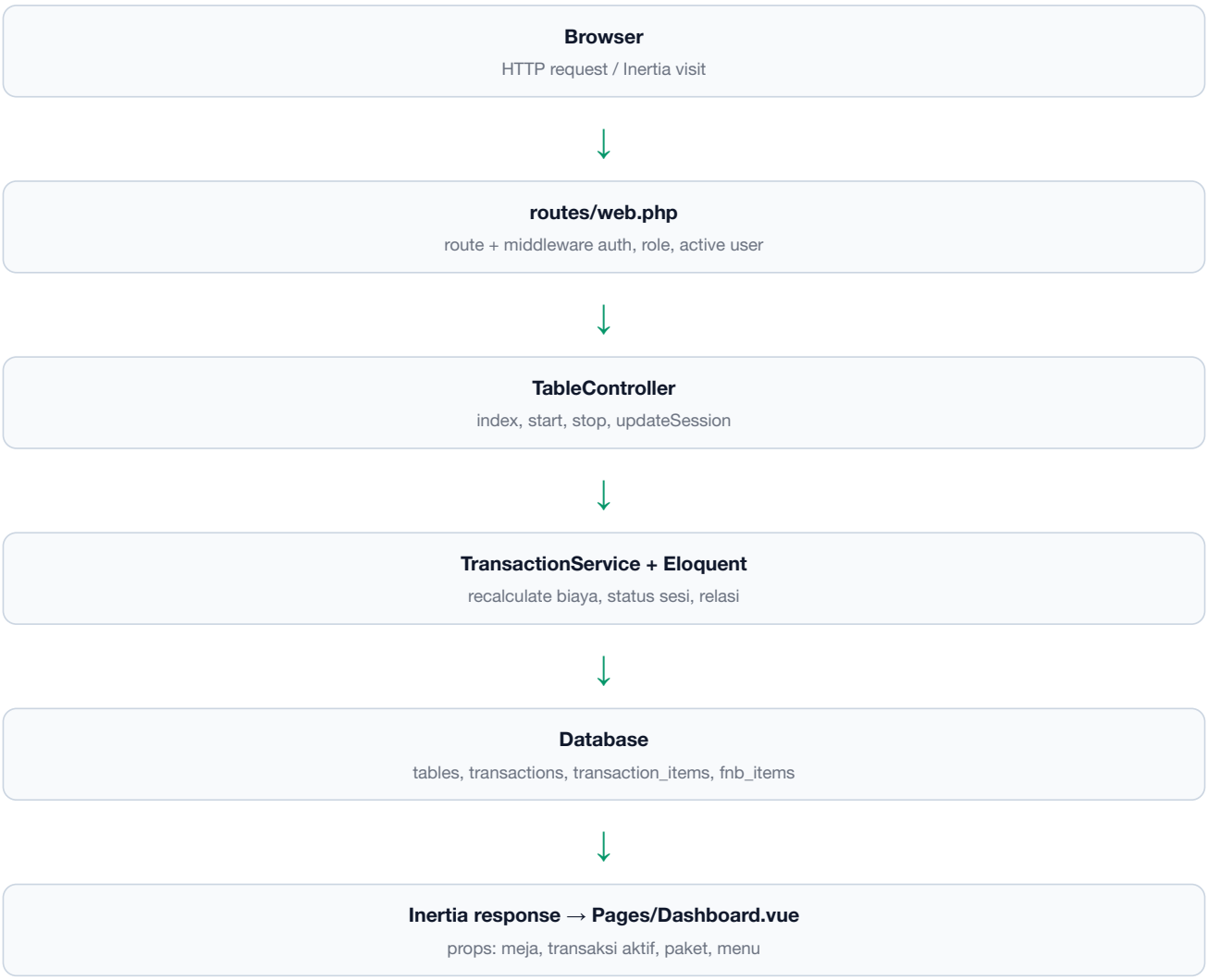
3.3 Tanggung jawab direktori

Folder	Tanggung jawab	Contoh berkas
`app/Http/Controllers`	Menerima request, memanggil service/model, mengembalikan response Inertia atau redirect.	`TableController.php`
`app/Http/Middleware`	Menjaga akses: role admin/kasir dan status pengguna aktif.	`CheckRole.php`
`app/Http/Requests`	Memusatkan aturan validasi form di luar controller.	`ProfileUpdateRequest.php`
`app/Models`	Relasi Eloquent, cast, accessor, dan konfigurasi model.	`Transaction.php`
`app/Services`	Aturan bisnis yang dipakai lintas controller; saat ini perhitungan biaya transaksi.	`TransactionService.php`
`app/Console/Commands`	Command artisan terjadwal untuk auto-checkout sesi kedaluwarsa.	`AutoCheckoutExpiredTables.php`
`database/migrations`	Riwayat perubahan skema; dapat dijalankan ulang dari nol.	18 berkas migrasi
`database/seeders`	Data awal: akun admin/kasir, 16 meja, paket tarif, dan menu F&B.	`DatabaseSeeder.php`
`resources/js/Pages`	Halaman Inertia yang dipetakan dari nama komponen di controller.	`Dashboard.vue`
`resources/js/Components`	Komponen UI yang dipakai ulang: modal, select, toast, tombol.	`BbSelect.vue`
`resources/scss`	Tema Bootstrap, variabel warna, dan kelas khusus berprefiks `bb-`.	`app.scss`
`tests`	Test fitur untuk auth, profil, transaksi, dan test unit dasar.	`TransactionServiceTest.php`

3.4 Alur request

Contoh: kasir membuka dashboard. Request melewati middleware autentikasi, masuk ke controller, membaca data meja dan transaksi aktif, lalu mengembalikan halaman Vue beserta props-nya. Karena controller hanya menyiapkan data dan Vue hanya merender props, perubahan tampilan tidak menyentuh aturan bisnis, dan sebaliknya.

> **Batas tanggung jawab.** Controller tidak menghitung biaya sendiri dan Vue tidak menjumlahkan total. Keduanya memakai hasil `TransactionService::recalculate()`, sehingga aturan uang hanya hidup di satu tempat dan dapat diuji terpisah.



Pola ini konsisten untuk halaman lain: route hanya mendefinisikan URL dan middleware, controller menyiapkan data, dan Vue tidak mengakses database secara langsung. Perhitungan uang tetap berada di server melalui `TransactionService::recalculate()`.

3.5 Struktur database

Tabel	Peran	Kunci / relasi
`users`	Akun admin dan kasir; login memakai username.	UUID; `role`, `is_active`
`tables`	Meja billiard dan channel relay.	UUID; `relay_channel`
`packages`	Paket tarif per jam (reguler siang/malam, VIP).	auto increment
`fnb_items`	Menu makanan/minuman, kategori, harga, gambar.	auto increment
`transactions`	Sesi billiard atau pesanan F&B mandiri.	`table_id` nullable, `user_id`
`transaction_items`	Baris pesanan F&B pada transaksi.	`transaction_id`, `fnb_item_id`
`sessions`	Session login dan payload Inertia.	`user_id`

Tabel	Peran	Kunci / relasi
`password_reset_tokens`	Token reset password; kolom `email` menyimpan username.	`email` (primary)

3.6 Konvensi penamaan

Elemen	Pola	Contoh
Controller	`XController`, method deskriptif	`FnbOrderController@checkout`
Model	singular, PascalCase	`TransactionItem`
Halaman Inertia	`Pages/Nama.vue`, mengikuti nama di controller	`Master/Reports/Revenue.vue`
Komponen	PascalCase, prefiks domain bila perlu	`BbSelect.vue`, `ConfirmModal.vue`
Route name	`resource.action`	`tables.start`, `reports.revenue`
Migrasi	`YYYY_MM_DD_HHMMSS_deskripsi`	`..._create_transactions_table.php`
Test	`NamaTest.php`, method `test_*`	`TransactionServiceTest`
CSS	kelas Bootstrap + prefiks `bb-`	`.bb-sidebar`, `.bb-card`

3.7 Halaman frontend

Kelompok	Halaman	Jumlah
Auth	Login, ForgotPassword, ResetPassword, ConfirmPassword	4
Operasional	Landing, Dashboard, Fnb/Index	3
Master & laporan	Admin/Index, Tables, Packages, FnbItems, Users, Reports/Analytics, FnbSales, Revenue, TableTransactions	9
Profil	Profile/Edit, Partials/UpdatePasswordForm	2

> **Catatan build.** `vite.config.js` memakai `base: '/poolstream/public/build/'`. Untuk deployment di root domain, ubah nilai tersebut menjadi `'/build/'`; untuk pengembangan lokal, gunakan `npm run dev` agar aset disajikan Vite.

3.8 Checklist struktur

Pemeriksaan	Bukti	Status
Logika HTTP di controller, bukan di route	`routes/web.php` hanya memetakan route dan middleware.	✓
Aturan biaya terpusat	`TransactionService::recalculate()` dipakai lintas alur transaksi.	✓

Pemeriksaan	Bukti	Status
Halaman Vue terkelompok	`Pages/Auth`, `Pages/Master`, `Pages/Reports`, `Pages/Profile`.	✓
Skema versioned	18 migrasi di `database/migrations`.	✓
Data uji tersedia	6 factory dan seeder data demo.	✓
Artefak build tidak dicampur sumber	Sumber di `resources/`, hasil di `public/build/`.	✓

3.9 Menelusuri satu perubahan

Contoh: menambahkan deskripsi pada paket tarif. Perubahan menyentuh enam lapisan dengan urutan yang dapat diprediksi, sehingga reviewer tahu berkas mana saja yang seharusnya berubah.

Lapisan	Berkas	Perubahan
Database	migration baru	Menambah kolom `description`
Model	`Package.php`	Menambah `description` ke fillable
Validasi	request class baru	Aturan nullable string
Controller	`PackageController.php`	Menyimpan dan mengirim props
Frontend	`Master/Packages.vue`	Input dan kolom tabel
Test	test class baru	Assertion create/update

3.10 Aturan penempatan berkas baru

- Logika request tetap di controller; logika bisnis yang dipakai lintas controller masuk ke `app/Services`.
- Validasi form masuk ke `app/Http/Requests`; otorisasi role dan status akun tetap di middleware.
- Halaman baru masuk ke `resources/js/Pages`; komponen yang dipakai ulang masuk ke `resources/js/Components`.
- Setiap perubahan skema memakai migration baru; data demo dan data uji dibuat lewat seeder/factory.

> **Ringkasan Bab 3.** Setiap lapisan memiliki batas tanggung jawab yang jelas: route memetakan akses, controller menyiapkan data, service menghitung aturan bisnis, model mengelola data, dan Vue merender tampilan. Struktur ini memudahkan review karena perubahan dapat dilacak ke folder yang tepat.

BAB 04 · WORKFLOW GIT

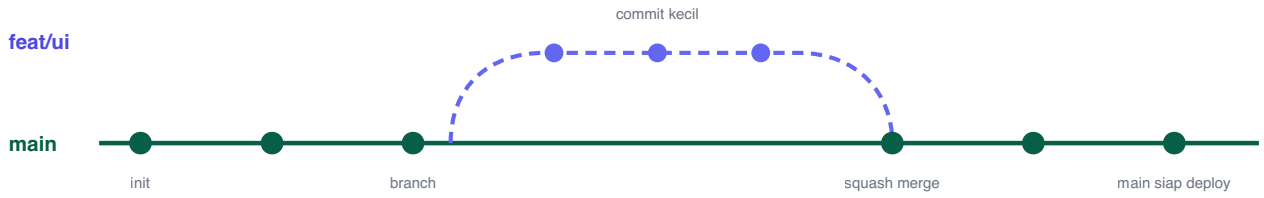
04

Branch, Commit, Review, dan Rilis

Workflow Git ditetapkan agar perubahan kecil mudah direview, riwayat mudah dibaca, dan branch utama tetap stabil. Aturan lengkapnya ada di `CONTRIBUTING.md`; bab ini merangkum model yang dipakai beserta bukti penerapannya di riwayat repository.

4.1 Model branching

Repository menggunakan model trunk-based: `main` adalah branch yang dapat di-deploy, sedangkan pekerjaan dilakukan di branch pendek yang segera di-merge kembali. Branch panjang seperti `develop` tidak digunakan karena menambah overhead integrasi tanpa manfaat untuk tim kecil.



Branch	Kegunaan	Contoh
`main`	Branch utama yang selalu stabil dan dapat di-deploy.	`main`
`feat/<scope>`	Fitur baru atau perubahan perilaku.	`feat/reports-export`
`fix/<scope>`	Perbaikan bug.	`fix/checkout-total`
`chore/<scope>`	Tooling, dependensi, refactor tanpa perubahan perilaku.	`chore/repo-hygiene`
`docs/<scope>`	Dokumentasi saja.	`docs/setup-guide`
`test/<scope>`	Penambahan atau perbaikan test.	`test/transaction-service`

Mengapa trunk-based? Tim kecil tidak perlu branch `develop` yang harus terus disinkronkan. Branch pendek mengurangi konflik, sementara `main` tetap stabil karena hanya menerima perubahan yang sudah lolos test dan review.

4.2 Siklus kerja

```
git switch main
git pull --ff-only origin main
git switch -c feat/reports-export

# kerjakan perubahan, commit dalam unit logis
git add <files>
git commit -m "feat(reports): add CSV export for revenue report"

# sinkronkan dengan main sebelum review
git fetch origin
git rebase origin/main
git push -u origin feat/reports-export
```

Setelah pull request dibuka, reviewer memeriksa deskripsi, diff, dan hasil test. Merge dilakukan dengan *squash merge* agar riwayat `main` tetap satu commit per perubahan, lalu branch dihapus. Untuk repository satu penulis, commit langsung ke `main` masih mungkin terjadi, tetapi perubahan baru tetap diarahkan ke alur branch dan pull request agar konsisten saat kolaborator bergabung.

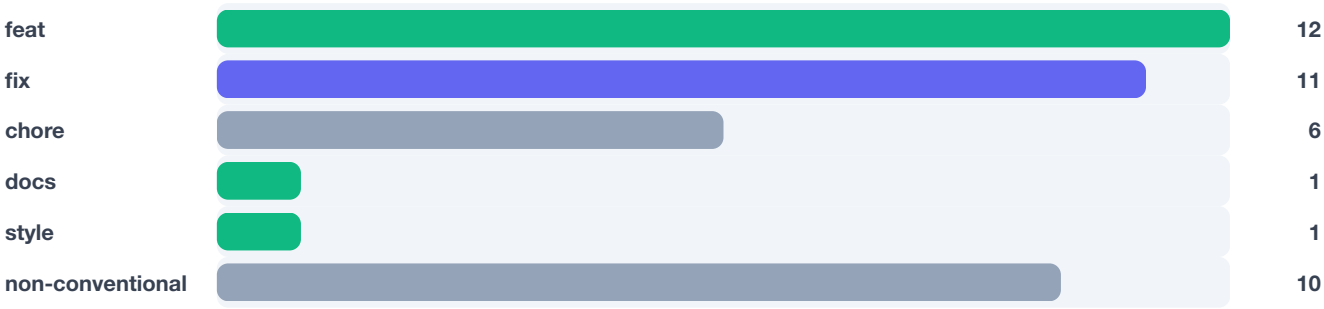
4.3 Conventional Commits

Format commit: `<type>(<scope>): <ringkasan imperatif>`. Ringkasan ditulis huruf kecil, tanpa titik akhir, dan menjelaskan *apa* yang berubah; body opsional menjelaskan *mengapa*.

Type	Makna	Contoh
`feat`	Fitur atau perilaku baru.	`feat(ui): add theme-matched SVG logo`
`fix`	Perbaikan bug.	`fix(auth): align password reset token column`
`docs`	Dokumentasi.	`docs: update AGENTS.md ...`
`style`	Formatting, tanpa perubahan perilaku.	`style: ...`
`chore`	Tooling, dependensi, build.	`chore(repo): stop tracking vendor and local env`

4.4 Distribusi commit

Dari 41 commit, 31 commit (75,6%) mengikuti Conventional Commits (Gambar 4.1). Sepuluh commit belum memakai convention: sembilan berasal dari periode awal sebelum aturan ditetapkan, satu commit dependency pada Oktober 2026 ditulis tanpa prefix.



4.5 Quality gate

Sebelum pull request dibuka, perubahan harus melewati pemeriksaan berikut:

Pemeriksaan	Perintah	Hasil saat ini
Test suite	`composer test`	19 passed, 48 assertions
Build frontend	`npm run build`	Berhasil (1,29 detik)
Gaya kode PHP	`./vendor/bin/pint --test`	Tool tersedia
Kebersihan Git	`git status --short`	Bersih

Checklist pull request di `CONTRIBUTING.md` juga mencakup: test untuk perilaku baru, screenshot untuk perubahan UI, tidak ada `.env` /kredensial, pembaruan `.env.example` bila ada variabel baru, serta migrasi/factory untuk perubahan database.

4.6 Commit terbaru

Hash	Tanggal	Pesan
`8ae9aa7`	28 Okt 2026	`fix(auth): align password reset token column with Laravel broker`
`25c40a8`	28 Okt 2026	`chore(repo): stop tracking vendor and local env`
`5e68c0c`	22 Okt 2026	`feat(ui): add theme-matched SVG logo`
`2aab567`	22 Okt 2026	`feat(ui): revamp profile page design`
`ae7672c`	20 Okt 2026	`feat(ui): revamp landing + kasir meja grid`
`d413c20`	20 Jul 2026	`fix: resolve ReportController ended_at vs end_time bug ...`

4.7 Rilis

Rilis ditandai dengan annotated tag SemVer pada commit `main` yang sudah lulus test dan memiliki build produksi. Repository belum memiliki tag; perintah berikut menjadi kontrak untuk rilis berikutnya.

```
git tag -a v1.0.0 -m "PoolStream v1.0.0"
git push origin v1.0.0
```

> **Kapan tag dibuat?** Tag dibuat dari commit di `main` yang sudah lulus test dan build, bukan dari branch fitur. Ini memastikan artefak rilis dapat direproduksi dari commit yang sama.

4.8 Aturan yang disepakati

Aturan	Penerapan
`main` tetap stabil	Tidak ada commit eksperimental langsung ke `main`; perubahan masuk melalui branch dan review.
Satu PR, satu tujuan	PR dibatasi pada satu fitur atau satu perbaikan agar diff mudah dibaca.
Commit atomik	Perubahan formatting, refactor, dan perilaku tidak dicampur dalam satu commit.
Rebase sebelum review	Branch disinkronkan dengan `main` menggunakan rebase, bukan merge commit.
Squash merge	Riwayat `main` menyimpan satu commit ringkas per perubahan.
Tag annotated	Rilis ditandai `vMAJOR.MINOR.PATCH` setelah test dan build lulus.

4.9 Contoh pull request

Judul PR mengikuti format commit; deskripsi menjelaskan perubahan, alasan, dan cara verifikasi.

```
Judul: feat(reports): add CSV export for revenue report

Perubahan
- Menambah endpoint export pada ReportController
- Menambah tombol Export di halaman Revenue.vue
- Menambah feature test untuk format CSV

Verifikasi
- composer test (19 passed, 48 assertions)
- npm run build (berhasil)
- git status --short (bersih)
```

4.10 Definisi selesai

Pemeriksaan	Bukti
Perubahan sesuai tujuan	Diff terbatas pada satu fitur atau satu perbaikan.
Test lulus	`composer test` hijau; test baru untuk perilaku baru.
Build lulus	`npm run build` berhasil.
Dokumentasi diperbarui	README dan `.env.example` bila ada variabel atau langkah baru.
Riwayat bersih	Tidak ada `.env`, kredensial, atau artefak build yang ikut.

Ringkasan Bab 4. Branch pendek, commit atomik, review PR, squash merge, dan tag annotated membentuk alur yang dapat diaudit. Aturan ini tidak hanya ditulis, tetapi tercermin pada 31 commit yang mengikuti Conventional Commits.

BAB 05 · DOKUMENTASI SETUP

05

Dari Repository Bersih ke Aplikasi Berjalan

Dokumentasi setup harus cukup untuk seseorang yang belum pernah melihat proyek ini. README menyediakan jalur cepat dan jalur manual; bab ini merangkum keduanya, termasuk konfigurasi environment, database, test, scheduler, dan troubleshooting yang paling sering muncul.

5.1 Prasyarat

Perangkat	Versi / keterangan	Dibutuhkan untuk
PHP	8.3 atau lebih baru; ekstensi `pdo_sqlite`, `pdo_mysql`, atau `pdo_pgsql` sesuai database.	Menjalankan Laravel dan test.
Composer	2.x.	Instalasi dependensi PHP dan script setup.
Node.js	20.19+ (disarankan LTS 22) dan npm.	Build Vite dan aset frontend.
Git	Versi terbaru.	Clone dan workflow branch.
Python	3.10+ dan `hidapi` (opsional).	Relay USB nyata; bridge saat ini masih dummy.
Database	SQLite (default), MySQL 8+, atau PostgreSQL 14+.	Penyimpanan data aplikasi.

5.2 Quick start

```
git clone https://github.com/anurr23/poolstream.git
cd poolstream
composer setup
composer dev
```

`composer setup` menjalankan seluruh langkah first-time setup secara berurutan:

#	Langkah	Efek
1	`composer install`	Memasang dependensi PHP dari `composer.lock`.
2	Copy `.env.example` → `.env` bila belum ada	Menyiapkan konfigurasi lokal tanpa menimpa file yang sudah ada.
3	Membuat `database/database.sqlite`	Database default siap dipakai.
4	`php artisan key:generate`	Mengisi `APP_KEY` untuk enkripsi session/cookie.
5	`php artisan migrate --force`	Menjalankan 18 migrasi skema.
6	`php artisan db:seed --force`	Membuat akun demo, 16 meja, paket, dan menu F&B.
7	`php artisan storage:link --force`	Menautkan `storage/app/public` ke `public/storage`.

#	Langkah	Efek
8	`npm install --ignore-scripts`	Memasang dependensi frontend dari `package-lock.json`.
9	`npm run build`	Mengompilasi SCSS dan JavaScript ke `public/build`.

Catatan. `composer setup` ditujukan untuk instalasi pertama. Menjalankannya ulang pada database yang sudah terisi dapat menimbulkan duplikasi data seed; gunakan `php artisan migrate:fresh --seed` hanya bila data lokal boleh dihapus.

5.3 Konfigurasi environment

Nilai berikut berasal dari `.env.example`. Kredensial nyata tidak dimasukkan ke repository.

Variabel	Default	Fungsi
`APP_NAME`	`PoolStream`	Nama aplikasi pada judul halaman.
`APP_ENV`	`local`	Mode environment Laravel.
`APP_URL`	`http://localhost`	Base URL untuk URL generator dan aset.
`DB_CONNECTION`	`sqlite`	Driver database: `sqlite`, `mysql`, atau `pgsql`.
`DB_DATABASE`	`database/database.sqlite`	Path/nama database.
`SESSION_DRIVER`	`database`	Session disimpan di tabel `sessions`.
`QUEUE_CONNECTION`	`database`	Antrean job disimpan di database.
`CACHE_STORE`	`database`	Cache disimpan di database.
`FILESYSTEM_DISK`	`local`	Disk default; upload publik memakai disk `public`.

5.4 Pilihan database

SQLite (default). Paling cepat untuk pengembangan dan test; tidak perlu server database terpisah.

```
touch database/database.sqlite
php artisan migrate --seed
```

MySQL. Buat database terlebih dahulu, lalu sesuaikan `.env`.

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=poolstream
DB_USERNAME=root
DB_PASSWORD=
```

PostgreSQL. Pastikan ekstensi `pdo_pgsql` aktif dan database sudah dibuat.

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=poolstream
DB_USERNAME=postgres
DB_PASSWORD=
```

5.5 Menjalankan aplikasi

`composer dev` menjalankan empat proses sekaligus dengan `concurrently` :

Proses	Perintah	Fungsi
Server	`php artisan serve`	HTTP server di `http://localhost:8000` .
Queue	`php artisan queue:listen`	Memproses job dari database queue.
Vite	`npm run dev`	Hot reload aset frontend.
Scheduler	`php artisan schedule:work`	Menjalankan auto-checkout setiap menit.

Buka `http://localhost:8000` , lalu login memakai **username**, bukan email.

Role	Username	Password	Akses
Admin	`admin`	`password`	Dashboard dan seluruh menu `/master/*` .
Kasir	`kasir`	`password`	Dashboard operasional dan pesanan F&B.

> **Wajib sebelum produksi.** Ganti password akun demo, set `APP_DEBUG=false` , dan gunakan kredensial database yang tidak dapat ditebak.

5.6 Setup manual

```
composer install
cp .env.example .env
php artisan key:generate

# SQLite: buat file database; MySQL/PostgreSQL: buat database lalu edit .env
touch database/database.sqlite

php artisan migrate --seed
php artisan storage:link
npm install
npm run build
php artisan serve
```

5.7 Testing

Test dijalankan terhadap SQLite in-memory (lihat `phpunit.xml`), sehingga tidak menyentuh database pengembangan.

```
composer test
# atau satu kelas tertentu
php artisan test --filter TransactionServiceTest
```

Test class	Cakupan	Method
`AuthenticationTest`	Login, logout, dan proteksi halaman.	4
`PasswordConfirmationTest`	Konfirmasi password sebelum aksi sensitif.	3
`PasswordResetTest`	Permintaan link dan reset password berbasis username.	4
`PasswordUpdateTest`	Perubahan password dari halaman profil.	2
`ProfileTest`	Tampilan dan pembaruan profil.	2
`TransactionServiceTest`	Rekalkulasi biaya F&B dan total transaksi.	1
`ExampleTest` (Feature/Unit)	Landing page, redirect guest, dan smoke test unit.	3

Hasil verifikasi: 19 test lulus dengan 48 assertion dalam 1,56 detik.

5.8 Scheduler dan queue

Command `tables:auto-checkout` menyelesaikan sesi yang melewati `expected_end_time` dan mematikan relay terkait. Command ini terdaftar di `bootstrap/app.php` dengan jadwal setiap menit.

```
# development: sudah termasuk composer dev
php artisan schedule:work

# produksi: cron Laravel
* * * * * cd /path/to/poolstream && php artisan schedule:run >> /dev/null 2>&1

# eksekusi manual
php artisan tables:auto-checkout
```

5.9 Relay hardware

Relay dipanggil dari PHP melalui `shell_exec` ke `app/Services/relay_controller.py`. Skrip yang ada saat ini adalah **dummy**: ia mengembalikan sukses tanpa mengakses perangkat. Implementasi `hidapi` asli masih dikomentari.

- Instal `hidapi` dan pastikan perangkat dikenali sistem sebelum mengaktifkan relay nyata.
- Verifikasi pemetaan `relay_channel` pada tabel `tables` agar lampu meja yang menyala sesuai.
- Uji dalam mode aman: matikan relay, mulai sesi, lalu cek perangkat sebelum dipakai operasional.

5.10 Troubleshooting

Gejala	Penanganan
`Vite manifest not found`	Jalankan `npm run build` atau `npm run dev`.
CSS/JS 404 setelah build	Base Vite adalah `/poolstream/public/build/`; sesuaikan `base` di `vite.config.js` atau path deployment.
`Database file does not exist`	Buat `database/database.sqlite` atau arahkan `DB_DATABASE` ke database yang ada.
`419 Page Expired`	Jalankan `php artisan config:clear` dan pastikan tabel `sessions` sudah dimigrasi.
Login gagal	Gunakan username `admin` / `kasir`; jalankan `php artisan db:seed` bila akun belum ada.
Relay tidak menyala	Skrip masih dummy; pasang `hidapi` dan ganti implementasi placeholder.
Scheduler tidak berjalan	Jalankan `php artisan schedule:work` atau pasang cron produksi.
Permission denied pada storage	`chmod -R ug+rwx storage bootstrap/cache` lalu ulangi `php artisan storage:link`.

5.11 Peta dokumentasi

Dokumen	Isi
`README.md`	Ringkasan aplikasi, fitur, stack, prasyarat, quick start, setup manual, test, scheduler, relay, struktur, troubleshooting.
`CONTRIBUTING.md`	Branching, Conventional Commits, checklist PR, hygiene, dan release tag.
`AGENTS.md`	Panduan teknis untuk agen/AI: arsitektur, domain model, konvensi, dan roadmap.
`documentation/`	Diagram instalasi relay dan gambar referensi perangkat.
Laporan PDF ini	Bukti pemenuhan tugas dan rekap verifikasi.

5.12 Berkas konfigurasi repository

Berkas	Fungsi
`.gitignore`	Mengabaikan `.env`, `vendor/`, `node_modules/`, cache, log, dan direktori IDE.
`.gitattributes`	Normalisasi line ending ke LF dan diff khusus untuk PHP, HTML, CSS, dan Markdown.
`.editorconfig`	UTF-8, LF, 4 spasi untuk PHP/JS/Vue, 2 spasi untuk YAML.
`.env.example`	Template environment tanpa kredensial nyata.
`composer.lock` / `package-lock.json`	Mengunci versi dependensi agar instalasi reproducible.

BAB 06 · VERIFIKASI

06 Bukti, Keterbatasan, dan Kesimpulan

Bagian ini merangkum bukti yang dapat diperiksa ulang, batas pengujian yang dilakukan, dan kesimpulan terhadap keempat kriteria tugas.

6.1 Checklist pemenuhan

Kriteria	Bukti utama	Status
Repository proyek	Repository publik, 41 commit, branch `main`, 1 contributor, riwayat 13 Jul–28 Okt 2026.	**Terpenuhi**
Struktur folder aplikasi	Pemisahan `app/`, `resources/js/`, `database/`, `routes/`, `tests/`; 231 berkas tracked; 18 halaman Vue.	**Terpenuhi**
Workflow Git	`CONTRIBUTING.md`, Conventional Commits 75,6%, branch naming, checklist PR, `.gitignore` / `.gitattributes` / `.editorconfig`.	**Terpenuhi**
Dokumentasi setup	README lengkap, setup script reproducible, test suite lulus, build frontend berhasil, troubleshooting.	**Terpenuhi**

6.2 Verifikasi teknis

Perintah	Hasil
`git rev-list --count HEAD`	41 commit.
`git ls-files | wc -l`	231 berkas tracked.
`git ls-files | grep -E '^(vendor/|\.env$)'`	Tidak ada output — bersih.
`php artisan test`	19 passed, 48 assertions, 1,56 detik.
`npm run build`	Berhasil, 1,29 detik.
`git fsck --no-dangling`	Tidak ada error integritas.
`git status --short`	Bersih setelah commit terakhir.

Ringkasan verifikasi. Repository dapat di-clone, dependensi dapat dipasang ulang dari lock file, test suite lulus, dan frontend dapat dibangun. Dokumentasi setup mencakup jalur cepat maupun manual.

6.3 Keterbatasan

- Test dijalankan pada container PHP 8.5; kebutuhan minimum proyek adalah PHP 8.3. Perbedaan minor antarversi PHP dapat memengaruhi hasil di environment lain.
- Relay hardware masih berupa placeholder; pengujian perangkat fisik belum dilakukan.
- Deployment produksi, konfigurasi web server, dan pengujian beban berada di luar ruang lingkup laporan ini.

- Riwayat Git lama masih menyimpan blob `.env`; rotasi kredensial dan pembersihan riwayat perlu dilakukan bila repository pernah memuat rahasia nyata.
- Cakupan test belum menyeluruh untuk seluruh kombinasi aturan bisnis dan peran pengguna.

6.4 Kesimpulan

Keempat kriteria tugas terpenuhi. Repository PoolStream memiliki riwayat yang dapat ditelusuri, struktur folder yang memisahkan backend, frontend, database, dan test, workflow Git yang terdokumentasi beserta quality gate, serta dokumentasi setup yang dapat diikuti dari repository bersih. Verifikasi teknis menunjukkan 19 test lulus dan build frontend berhasil.

Perbaikan hygiene repository—mengeluarkan `vendor/` dan `.env` dari indeks Git—menjadikan repository lebih ringan, aman, dan sesuai praktik standar Laravel. Dengan demikian, repository tidak hanya memenuhi syarat administratif, tetapi juga siap digunakan sebagai basis pengembangan berikutnya.

6.5 Rekomendasi

- Aktifkan branch protection untuk `main` dan wajibkan pull request serta status check test.
- Tambahkan CI (GitHub Actions) untuk menjalankan `composer test`, `npm run build`, dan `pint --test` pada setiap PR.
- Rotasi `APP_KEY` dan kredensial database, lalu bersihkan blob `.env` dari riwayat bila diperlukan.
- Ganti relay dummy dengan implementasi `hidapi` dan uji perangkat sebelum operasional.
- Tambahkan audit log untuk aksi kritis (start, stop, cancel, adjustment) sesuai roadmap di `AGENTS.md`.

DISUSUN OLEH	DIPERIKSA OLEH
Muhammad Anurohim	

LAMPIRAN A

Perintah Penting

A

Daftar perintah yang paling sering dipakai saat mengembangkan dan memverifikasi proyek.

Kebutuhan	Perintah
Instalasi pertama	`composer setup`
Menjalankan stack development	`composer dev`
Menjalankan seluruh test	`composer test`
Menjalankan satu test	`php artisan test --filter TransactionServiceTest`
Migrasi dan seed	`php artisan migrate --seed`
Reset database lokal	`php artisan migrate:fresh --seed`
Auto-checkout manual	`php artisan tables:auto-checkout`
Scheduler development	`php artisan schedule:work`
Build frontend	`npm run build`
Vite development	`npm run dev`
Format PHP	`./vendor/bin/pint`
Cek gaya PHP	`./vendor/bin/pint --test`
Daftar route	`php artisan route:list`
Bersihkan config/cache	`php artisan optimize:clear`
Lihat log	`tail -f storage/logs/laravel.log`

Referensi. Setup lengkap: `README.md`. Workflow Git: `CONTRIBUTING.md`. Panduan teknis agen: `AGENTS.md`. Repository: `github.com/anurr23/poolstream`.